

AsystomAdvisor API

Abstract

This document is the User Guide of the AsystomAdvisor Application Programming Interface.

This API allows to get data received from Asystom Sentinel devices and produced in the Asystom back-end. It also allows to create and update dashboards, to change a beacon settings and to list all devices part of a Grafana organisation.

Changelog

Version number	Written by	Revision date	Note
1.0	Piotr Diop	27/02/2019	Initial draft
1.1	Piotr Diop	22/07/2020	Rewrote .md for better LaTeX / pdf integration
1.2	Piotr Diop	07/09/2020	Added chapter for resolving long jobs
1.3	Piotr Diop	09/02/2020	Updated long job resolution for V4 arch
1.4	Piotr Diop	28/04/2022	Added chapter for FFT Zoom support & Merge with PUSH API
1.5	Piotr Diop	10/06/2022	Added Anomaly Detector support
1.6	Piotr Diop	27/03/2023	Added support for alert-related queries (AlertList, AlertHistory)
1.7	Piotr Diop	03/08/2023	Added support for training-related queries (TrainingSessionList, BeaconVersionHistory)
1.8	Piotr Diop	04/09/2023	Added "All" query support
1.9	Piotr Diop	18/09/2023	Renamed back "Battery" to "Batt_Lvl", and removed "Humidity"
2.0	Piotr Diop	09/10/2023	Added "tags" and "analysis_profile" to available IdCard fields
2.1	Patrick Duc	30/05/2024	Added a chapter for Update API
2.2	Patrick Duc	14/06/2024	Added a description for beacon update and beacon settings change
2.3	Patrick Duc	28/06/2024	Added table of contents
2.4	Patrick Duc	15/07/2024	Fixed name of beacon settings selection part argument for beacon settings change request
2.5	Patrick Duc	05/08/2024	Updated description of beacon settings change service and added analysis_profile parameter
2.6	Patrick Duc	04/10/2024	Added description of services allowing to access Signature* information.
2.7	Patrick Duc	23/10/2024	Added description of new service /devices.

Version number	Written by	Revision date	Note
2.8	Patrick Duc	22/01/2025	Added description of new parameter to /IdCard service.
2.9	Patrick Duc	11/02/2025	Improved document description, removed useless parts and missing links.
2.10	Patrick Duc	19/06/2025	Fixed some typos and slightly improved presentation
2.11	David Latil	16/07/2025	Added description of anomaly score trend fields
2.12	Patrick Duc	18/09/2025	Updated the service prefix to /dev/api
2.13	Patrick Duc	06/11/2025	Updated description of long job return codes and OAuth2-specific URL path prefix
2.14	Patrick Duc	30/01/2026	Added description of new services (fetch all data from all devices, get identifier of the device attached to a dashboard, get dashboard information by name)
2.15	Patrick Duc	19/05/2026	Added description of new service DELETE /dashboard. Added a link to the Asystem tool to generate hex strings representing beacon settings. Removed invalid use of Content-Type header in GET and DELETE HTTP requests.

Table of contents

IMPORTANT NOTICE	5
PARAMETER TYPES	6
Read-only API	7
General Information	7
Status codes and responses	7
Data fetch requests	8
GET data with “last” parameter key.....	8
GET data with “from” and “to” keys.....	9
GET “All” measurements for a given device	10
GET “All” measurements for all devices.....	12
GET Identity Card for a given Beacon	13
GET Identity Card for a given beacon, including short ISO classification name.....	14
GET all registered devices and their associated machine names.....	14
GET list of all anomaly classes (Incident Advisor)	15
GET list of all current alert states	15
GET alert state changes for a given device.....	16
GET list of training sessions for a given device	17
GET software version history for each registered device	18
Accessing Signature data	18
Accessible parameters	18
Signature types	19
Signature	19
Signature settings	20
Signature_SpectrumZoom	21
Signature_Anomaly	22
Processing very long jobs	26
Response when the long job is still pending	26
Response when the long job has completed.....	26
List all devices per organization	27
Get the identifier of a device attached to a dashboard.....	29
Get device information by its identifier	30
UPDATE API.....	32
General information	32

Services..... 32

 Dashboard creation..... 32

 Dashboard update 35

 Dashboard deletion..... 39

 Dashboard settings change 40

IMPORTANT NOTICE

The Advisor API provides two ways to authorize a request :

- HTTP Basic Authentication
- OAuth2 authorization.

The latter is a largely improved way to authorize API requests.

The URL path of API requests described in this document are introduced by the prefix /dev. This prefix applies to all requests.

For instance, this a valid API request :

```
curl https://<domain>/dev/api?datatype=Temperature&device_mac=00-00-00-00-00-00-00-00-01&from=2023-02-27T00:46:17Z&to=2023-02-27T01:49:15Z
--header "Authorization: Basic <valid authentication string>"
--request GET
```

However, if OAuth2 authorization is to be activated on a customer site, the prefix /dev MUST be replaced by /dev/api for API requests to be taken into account.

If not, a 404 Not Found status code is returned.

Please note that data fetch requests described in the Read-only API also use /api as endpoint identifier. Consequently, using OAuth2 authorization, the path of such requests becomes /dev/api/api.

This change of prefix is made necessary for the API to remain compatible with both HTTP Basic Authentication and OAuth2 authorization.

In the following, it is assumed that OAuth2 authorization has not been activated, and therefore the prefix used is /dev.

PARAMETER TYPES

Beacons compute monitoring indicators either on a pre-defined periodic schedule or based on events (e.g. vibration level). All indicator values are timestamped.

All indicators can be fetched using one of two key parameters:

- `last` key allows fetching the most recent entries for an indicator
- `from` and `to` keys allow fetching a particular group of entries between two dates.

In addition to indicator related data, a user can also fetch meta-information available inside our web interface (AsystomAdvisor). Examples of such meta-information are “Identity card”, “Device List”, “Anomaly List”, “Alert List” or “Alert History”. See next chapters for more details.

Read-only API

General Information

URL : `https://<domain>/dev/<endpoint>/`

Method : GET

Authentication type : HTTP Basic Authentication

Data Constraints	Description
Date fields	ISO 6801 specification [YYYY-MM-DDTHH-MM-SSZ]

Queryable parameters	Description
<indicator name>	One of: {"Batt_Lvl", "Temperature", "Signature", "Signature_Settings", "Signature_SpectrumZoom", "Signature_Anomaly", "All"}
IdCard	Identity card for the beacon & monitored machine
DeviceList	Addressable devices and their associated machines
AnomalyList	Fault category lookup for Incident Advisor feature
AlertList	Lists current alerts for all existing devices
AlertHistory	Lists current and past alerts for a given device
TrainingSessionList	Lists all training sessions for a given device
BeaconVersionHistory	Lists current software version for each registered device

Status codes and responses

Request Failed

- **Execution Error Code :** 500 Internal Server Error
- **Payload Content :** <error message>

Resource already taken / no longer available

- **Execution Error Code :** 410 Gone
- **Payload Content :** { "error": "This job has either completed or does not exist anymore" }

Request Successful

- **Success Code** : 200 OK
- **Payload Content** : All requested datapoints OR the status of a resource under construction (for more details see chapter “2.3 Processing very long jobs”).

In the following example, we receive all temperature points for device 00-00-00-00-00-00-01 between 2023-02-27T00:46:17Z and 2023-02-27T01:49:15Z.

Request

```
curl https://<domain>/dev/api?datatype=Temperature&device_mac=00-00-00-00-00-00-01&from=2023-02-27T00:46:17Z&to=2023-02-27T01:49:15Z
--header "Authorization: Basic <valid authentication string>"
--request GET
```

Response

```
[
  {
    "time": "2023-02-27T00:49:14.342Z",
    "value": 23.762550545510067
  },
  {
    "time": "2023-02-27T01:18:17.159Z",
    "value": 23.935980773632423
  }
]
```

Request accepted for further processing (applies only to very long jobs)

- **Success Code** : 202 Accepted
- **Payload Content** : The location of the resource that is currently under construction.

```
{
  "location": "01010101-efefef",
  "datatype": "JobQueue"
}
```

Data fetch requests

GET data with “last” parameter key

Keys	Values
device_mac	00-00-00-00-00-00-01
datatype	Temperature
last	3

Request

```
curl https://<domain>/dev/api?datatype=Temperature&device_mac=00-00-00-00-00-00-00-01&last=3
--header "Authorization: Basic <valid authentication string>"
--request GET
```

Response

200 OK

```
[
  {
    "time": "2019-02-27T00:49:14.342Z",
    "value": 23.762550545510067
  },
  {
    "time": "2019-02-27T00:47:14.735Z",
    "value": 23.866608682383458
  },
  {
    "time": "2019-02-27T00:46:17.159Z",
    "value": 23.935980773632423
  }
]
```

GET data with “from” and “to” keys

Keys	Values
device_mac	00-00-00-00-00-00-00-01
datatype	Batt_Lvl
from	2023-02-27T00:40:07Z
to	2023-02-28T14:46:07Z

Request

```
curl https://<domain>/dev/api?datatype=Batt_Lvl&device_mac=00-00-00-00-00-00-00-01&from=2023-02-27T00:40:07Z&to=2023-02-28T14:46:07Z
--header "Authorization: Basic <valid authentication string>"
--request GET
```

Response

200 OK

```
[
  {
    "time": "2019-02-27T00:47:14.735Z",
    "value": 1.8112458991378653
  },
  {
    "time": "2019-02-27T00:49:14.342Z",

```

```

        "value": 1.7730983443961241
    }
]

```

GET “All” measurements for a given device

Keys	Values
device_mac	00-00-00-00-00-00-01
datatype	All
last	2

This API will return all available measurements associated to a device. The response format is designed to convey time-series data, where each data point consists of a timestamp and a corresponding (set of) value(s). Each measurement is defined by three fields:

- **name** (string): This field represents the name of the measurement
- **columns** (array of strings): This field contains an array of column names. Each column name represents a specific attribute or property associated with the measurement.
- **values** (array of arrays): This field contains an array of data points. Each data point is represented as an array, where the order of elements corresponds to the columns defined earlier.

This API is also compatible with `from` and `to` keys instead of `last`.

The response in the example below has been abridged due to its excessive size.

Request

```

curl https://<domain>/dev/api?datatype=All&device_mac=00-00-00-00-00-00-00-00-01&last=2
    --header "Authorization: Basic <valid authentication string>"
    --request GET

```

Response

200 OK

```

[
  {
    "name": "Temperature",
    "columns": [
      "time",
      "value"
    ],
    "values": [
      [
        "2022-11-15T07:49:50.69Z",
        22.513852903028976
      ]
    ]
  }
]

```

```

        [
            "2022-11-15T07:48:50.651Z",
            22.479166857404437
        ]
    ],
    ...,
    ...,
    ...,
    {
        "name": "Signature",
        "columns": [
            "time",
            "s_00",
            ...,
            "sonic_rmslog"
        ],
        "values": [
            [
                "2022-11-15T07:48:50.651Z",
                -76.14785992217898,
                ...,
                -76.55985351338978
            ],
            [
                "2022-11-15T07:46:50.694Z",
                -75.77248798352025,
                ...,
                -76.35156786449988
            ]
        ]
    },
    ...,
    {
        "name": "Signature_Anomaly",
        "columns": [
            "time",
            "model",
            ...,
            "aux_f2"
        ],
        "values": [
            [
                "2022-11-15T07:48:50.651000064Z",
                "1.8.2",
                ...,
                null
            ],
            [
                "2022-11-15T07:46:50.694000128Z",
                "1.8.2",

```

```

        ...
        null
    ]
]
}
]

```

GET “All” measurements for all devices

Keys	Values
device_mac	All
datatype	All
last	2

This API will return all available measurements for all devices. The response format is the same as when requesting all data for a specific device, except that the returned columns include a device information in addition to time and value that shows the device DevEUI corresponding to the value.

This API is also compatible with `from` and `to` keys instead of `last`.

The response in the example below has been abridged due to its excessive size.

Request

```

curl https://<domain>/dev/api?datatype=All&device_mac=All& from=2025-01-01T00:00:00.000Z&to=2025-01-05T00:06:00.000Z
--header "Authorization: Basic <valid authentication string>"
--request GET

```

Response

200 OK

```

[
  {
    "name": "Batt_Lvl",
    "columns": [
      "time",
      "value",
      "device"
    ],
    "values": [
      [
        "2025-01-01T00:00:17.479Z",
        3.2852674143587395,
        "00-04-a3-0b-01-08-2d-6a"
      ],
      [
        "2025-01-01T00:00:17.479Z",
        3.150988021667811,

```

```

        "00-04-a3-0b-00-e9-9d-d3"
    ],
    [
        "2025-01-01T00:00:17.479Z",
        2.98466468299382,
        "00-04-a3-0b-01-06-5e-ab"
    ],
    ...
]
},
...
}
]

```

GET Identity Card for a given Beacon

Keys	Values
device_mac	00-00-00-00-00-00-00-01
datatype	IdCard

Request

```

curl https://<domain>/dev/api?datatype=IdCard&device_mac=00-00-00-00-00-00-00-01
--header "Authorization: Basic <valid authentication string>"
--request GET

```

Response

```

200 OK

{
    "machine_name": "Demo Machine 01",
    "mac_address": "00-00-00-00-00-00-00-01",
    "process_function": "Compressor",
    "sensor_location": "Input Shaft",
    "sensor_orientation": "Z↑ Y↻",
    "manufacturer": "ABC01",
    "model": "012345",
    "type": "Motor",
    "subtype": "Brushed",
    "class": "Class II",
    "tags": "Machine",
    "analysis_profile": "Rotating"
}

```

Please note that the returned information includes the long (detailed) name of the ISO machine classification. The short ISO classification name may also be returned if the query parameter `both_iso_class_names` is set with the value `true`.

Here is an example of use.

GET Identity Card for a given beacon, including short ISO classification name

Keys	Values
device_mac	01-02-03-04-05-06-07-08
datatype	IdCard

Request

```
curl https://<domain>/dev/api?datatype=IdCard&device_mac=01-02-03-04-05-06-07-08&both_iso_class_names=true
--header "Authorization: Basic <valid authentication string>"
--request GET
```

Response

200 OK

```
{
  "machine_name": "Demo Machine 01",
  "mac_address": "00-00-00-00-00-00-01",
  "process_function": "Compressor",
  "sensor_location": "Input Shaft",
  "sensor_orientation": "Z↑ Y↻",
  "manufacturer": "ABC01",
  "model": "012345",
  "type": "Motor",
  "subtype": "Brushed",
  "class": "Rotodynamics pumps > 1kW | Category II <= 200kW",
  "tags": "Machine",
  "analysis_profile": "Rotating",
  "machine_class_shortcode": "b3"
}
```

GET all registered devices and their associated machine names

Keys	Values
datatype	DeviceList

Request

```
curl https://<domain>/dev/api?datatype=DeviceList
--header "Authorization: Basic <valid authentication string>"
--request GET
```

Response

200 OK

```
[
  {
    "Name": "Demo Machine 01",
    "MacAddr": "00-00-00-00-00-00-01"
```

```

    },
    {
      "Name": "Demo Machine 02",
      "MacAddr": "00-00-00-00-00-00-02"
    }
  ]

```

GET list of all anomaly classes (Incident Advisor)

Keys	Values
datatype	AnomalyList
lang	EN

Request

```

curl https://<domain>/dev/api?datatype=AnomalyList
  --header "Authorization: Basic <valid authentication string>"
  --request GET

```

Request

200 OK

```

[
  {
    "strategy": "0X101",
    "model": "1.8.4",
    "content": "{ \"strategy\": [{ \"name\": \"Ultrasound\", \"class\": \"Fault Category 0\", \"content\": { \"causes\": [\"Early stage of rolling element bearing wear\"...]} }"
  },
  ...
]

```

Currently, EN and FR languages are supported.

Further information on anomaly classes and their usage can be found in « REST API - internals » document

GET list of all current alert states

Keys	Values
datatype	AlertList

Request

```

curl https://<domain>/dev/api?datatype=AlertList
  --header "Authorization: Basic <valid authentication string>"
  --request GET

```

Response

200 OK

```
[
  {
    "alertType": "ANOMALY SCORE",
    "state": "no_data",
    "time": "2023-02-09T17:27:22Z",
    "url": "/monitor/d/a0123456/dashboard_1"
  },
  {
    "alertType": "BATTERY",
    "state": "alerting",
    "time": "2023-03-24T14:23:10Z",
    "value": 2.9968719005111772,
    "url": "/monitor/d/b456789/dashboard_2"
  }
]
```

There are three possible states for each alert: “*ok*”, “*alerting*” and “*no_data*”.

When an alert is in “*alerting*” state, the “*value*” field indicates the value that caused the alert to trigger.

GET alert state changes for a given device

Keys	Values
datatype	AlertHistory
device_mac	00-00-00-00-00-00-01
last	4

Request

```
curl https://<domain>/dev/api?datatype=AlertHistory&device_mac=00-00-00-00-00-00-01&last=4
--header "Authorization: Basic <valid authentication string>"
--request GET
```

Response

200 OK

```
[
  {
    "time": "2023-03-24T09:15:18.938Z",
    "alertName": "BATTERY",
    "state": "ok"
  },
  {
    "time": "2023-03-24T00:20:42.916Z",
    "alertName": "BATTERY",
    "state": "no_data"
  },
]
```

```

{
  "time": "2023-03-22T14:04:32.923Z",
  "alertName": "VIBRATORY SEVERITY",
  "state": "ok"
},
{
  "time": "2023-03-21T22:00:24.920Z",
  "alertName": "VIBRATORY SEVERITY",
  "state": "alerting",
  "value": 9.748989089799343
}
]

```

This API is also compatible with `from` and `to` keys instead of `last`.

GET list of training sessions for a given device

Keys	Values
<code>datatype</code>	<code>TrainingSessionList</code>
<code>device_mac</code>	<code>00-00-00-00-00-00-01</code>

Request

```

curl https://<domain>/dev/api?datatype=TrainingSessionList&device_mac=00-00-00-00-00-00-01
  --header 'Authorization: Basic <valid authentication string>'
  --request GET

```

Response

200 OK

```

{
  "training_sessions": {
    "364": {
      "0": {
        "start": "2023-05-09T21:18:00.000Z",
        "end": "2023-05-10T16:02:00.000Z"
      }
    },
    "368": {
      "0": {
        "start": "2023-05-15T15:52:00.000Z",
        "end": "2023-05-15T18:49:00.000Z"
      },
      "1": {
        "start": "2023-05-12T18:10:00.000Z",
        "end": "2023-05-14T17:10:00.000Z"
      }
    }
  }
},

```

```
"mac_address": "00-00-00-00-00-00-00-01"
}
```

GET software version history for each registered device

Keys	Values
datatype	BeaconVersionHistory

Request

```
curl https://<domain>/dev/api?datatype=BeaconVersionHistory
--header "Authorization: Basic <valid authentication string>"
--request GET
```

Response

200 OK

```
[
  {
    "time": "2020-12-18T10:15:39.614Z",
    "device": "00-00-00-00-00-00-00-01",
    "version": "v4.40"
  },
  {
    "time": "2021-07-13T09:59:30.234Z",
    "device": "00-00-00-00-00-00-00-02",
    "version": "v4.36"
  }
]
```

Accessing Signature data

The services listed in this section allow to retrieve signature data.

Accessible parameters

Numerical values are all IEEE-754 64bit floating points.

Signature type parameters are arrays of multiple values.

Parameter Name	Min value	Max value	Notes
Temperature	-273.15 °C	2000 °C	
Batt_Lvl	0 V	-	Batteries must be replaced before running below 2.1V

Current_Loop	0 A	30 A	Typically used with a mA process clamp meter (4-20mA)
Pressure	0 Pa	2000 Pa	
PT100	-273.15 °C	2000 °C	Platinum Resistance Thermometer
TC	-273.15 °C	2000 °C	Thermocouple
Signature*	N/A	N/A	<i>byte array</i>
Signature_Settings*	N/A	N/A	<i>byte array</i>
Signature_SpectrumZoom*	N/A	N/A	<i>byte array</i>
Signature_Anomaly*	N/A	N/A	<i>byte array</i>

* Detailed description of Signature content is presented in next chapter

Signature types

Signature and **Signature_*** are arrays of different indicators related to vibration and ultrasonic measurements. They can be split in two groups:

Measurements Group :

Signature: The current values for *Signature* vector.

Signature_SpectrumZoom: The current values for *FFT Zoom* vector. Its size/decoding is entirely dependent on the corresponding Signature_Settings values (See Settings Group).

Signature_Anomaly: The current values for *Incident Advisor* related variables.

Settings Group :

Signature_Settings: Configuration model defining expected signature contents. Currently, it is only used for FFT Zoom feature.

Signature

This measurement exposes the following values:

Parameter		
Name	Unit	Detail

s_00..s_09 *	dB	10 vibration frequency bands s_00 to s_09, linearly spaced between 0 and 2kHz, 0dB represents the sensor full scale equivalent to 16g. Bands are defined as follows: {[0-200], [200-400], [400-600], [600-800], [800-1k], [1k-1.2k], [1.2k-1.4k], [1.4k-1.6k], [1.6k-1.8k], [1.8k-2k]} (Hz)
s_10..s_19	dB	10 sound frequency bands s_10 to s_19, linearly spaced between 0 and 80kHz. 0dB represents the sensor full scale equivalent to 120dB SPL. Bands are defined as follows: {[0-8k], [8k-16k], [16k-24k], [24k-32k], [32k-40k], [40k-48k], [48k-56k], [56k-64k], [64k-72k], [72k-80k]} (Hz)
sonic_rmslog	dB	RMS value for ultrasonic data in the ultrasonic band [30kHz-80kHz]
vib_x_vel	mm/s	X-Axis: RMS level of vibration velocity
vib_x_acc	g	X-Axis: RMS acceleration value
vib_x_peak	g	X-Axis: absolute peak acceleration level
vib_x_kurt	-	X-Axis: kurtosis (reflects the shape of the acceleration amplitude probability distribution)
vib_x_root		RPM X-Axis: Root frequency(expressed in RPM, typically the speed of a rotating machine)
vib_x_f1	mm/s	X-Axis: RMS velocity (around the root frequency)
vib_x_f2	mm/s	X-Axis: RMS velocity (around the 1st harmonic)
vib_x_f3	mm/s	X-Axis: RMS velocity (around the 2nd harmonic)
vib_y_[..]	-	Y-Axis follows the same description as X-axis for the above parameters
vib_z_[..]	-	Z-Axis follows the same description as X-axis for the above parameters
temp	°C	Surface temperature as measured at the accelerometer level

(*) Field is subject to **[Preferred Axis]** setting in Asystom's visualization tool. It typically allows to select a specific vibration axis for the analysis.

Signature settings

The contents of this measurement define the type and formatting of alternate Signature types. Currently, the only alternative to the classic signature is the FFT Zoom feature.

Signature_Settings is a hexadecimal array with elements encoded in Little Endian. It is a 26-byte array formatted with the following order:

Parameter	Min	Max	Size	Usage/Possible Values
-----------	-----	-----	------	-----------------------

handle	0	254	1 Byte	Server-side generated handle. Corresponding FFT Zoom values shall be read in regards to this parameter. Loops over 254.
mode	0	2	1 Byte	0: Inactive, 1: every N signatures, 2: burst of size N (See next parameter to define N)
num_steps	0	254	1 Byte	Refers to steps number N used above
algorithm	0	0	1 Byte	0: FFT Zoom (Only FFT zoom is supported for now)
sensor_sel	1	12	1 Byte	Sensor selection. 3: Accelerometer, 12: Microphone
Orientation	0	3	1 byte	0: No preferred orientation (axes combined), 1: X axis, 2: Y axis, 3: Z axis
p_1	0	*	4 Bytes	Upper bound for the FFT zoom window (in Hz). Refer to sensor specification for current limits and recommendations
p_2	0	*	4 Bytes	Lower bound for the FFT zoom window (in Hz). Refer to sensor specification for current limits and recommendations
p_3	0	4	4 Bytes	Compression Type. 0: 50 bins (8 bit), 1: 50 bins (16 bits), 2: 100 bins (8bit), 3: 100 bins (16 bit), 4: 200 bins (8 bit)
p_4 *	0	5	4 Bytes	Spectrum Type: 1: RMS, 2: Peak, 3: RMS velocity, 4: Peak velocity, 5: RMS envelope, 6: Peak envelope
p_5 *	0	*	4 Bytes	High pass filter cutoff frequency. Only used by envelope spectrum. Refer to sensor specification for current limits and recommendations

(*) These parameters are supported only for beacons with version > 4.40 NOTE: Spectrum types 3 to 6 are valid for vibration sensor only.

Signature_SpectrumZoom

This measurement exposes the values related to the FFT Zoom feature. Its structure is as follows:

Subfield	Unit	Detail
----------	------	--------

handle	-	To be used as a lookup handle to find the appropriate settings that will define the contents of bin_* elements
bin_0000..bin_0199 dBFS		The number of populated bins depends on the compression_type parameter. Currently, can be either 50, 100 and 200. The length for each bin is linearly deducted from the upper/lower bounds of the FFT Zoom window and the number of expected bins. 0dB represents the full scale for the sensor

Signature_Anomaly

This measurement exposes values computed by the Incident Advisor/Anomaly Detector modules. Its structure is as follows:

Subfield	Unit	Detail
model	-	Anomaly Detector version number.
strategy	-	Hex value indicating which interpretation shall be applied to the values tagged as 'adv_[.].similarity'
score_anomaly	-	Anomaly Score, a percentage expressed between [0..1]
score_anomaly_trend	-	Anomaly Score trend, calculated only for ON states
score_anomaly_trend_unfiltered	-	Anomaly Score trend, calculated regardless of the state
aux_f0	-	0 if the machine is evaluated as being OFF (not in one of its operating states), 1 when the machine is ON. The fields below can be discarded when the machine is considered OFF
aux_f1	mm/s	Current vibratory severity
aux_f2	RPM	Current RPM value. Only applies to rotating machines
aux_f3	-	Unitless shock level. Ranging from 0 (no shock) to 100 (extreme shock level).
adv_[...].similarity	-	Multiple consecutive values tagged adv_0_similarity, adv_1_similarity.. up to adv_n_similarity expressing the fault class similarity. This is a percentage expressed between [0..1] for the corresponding fault category. Negative, or null values shall be discarded.

Applicable fields per strategy is shown below.

Complete lookup is accessible using the “**AnomalyList**” API (“*GET list of all anomaly classes (Incident Advisor)*”).

	Rotating Profile for beacon version < 4.38	Rotating Profile for beacon version > 4.38	Non Rotating Profile for beacon version < 4.38	Non Rotating Profile for beacon version > 4.38	Valve Profile
strategy	0x101	0x102	0x201	0x202	0x301 / 0x302
model	Machine learning model version number	Machine learning model version number	Machine learning model version number	Machine learning model version number	Machine learning model version number
score_anomaly	Anomaly Score expressed between [0..1]	Anomaly Score expressed between [0..1]	Anomaly Score expressed between [0..1]	Anomaly Score expressed between [0..1]	
aux_f0	0 if the machine is evaluated as being OFF (not in one of its operating states), 1 when the machine is ON. The fields below can be discarded when the machine is considered OFF	0 if the machine is evaluated as being OFF (not in one of its operating states), 1 when the machine is ON. The fields below can be discarded when the machine is considered OFF	0 if the machine is evaluated as being OFF (not in one of its operating states), 1 when the machine is ON. The fields below can be discarded when the machine is considered OFF	0 if the machine is evaluated as being OFF (not in one of its operating states), 1 when the machine is ON. The fields below can be discarded when the machine is considered OFF	
aux_f1	Current vibratory severity (mm/s)	Current vibratory severity (mm/s)	Current vibratory severity (mm/s)	Current vibratory severity (mm/s)	Current vibratory severity (mm/s)
aux_f2	Current RPM value (round per minute)	Current RPM value (round per minute)			
aux_f3	Shock indicator (unitless ranging in [0;100])	Shock indicator (unitless ranging in [0;100])	Shock indicator (unitless ranging in [0;100])	Shock indicator (unitless ranging in [0;100])	Shock indicator (unitless ranging in [0;100])
adv_0_similarity *	Ultrasound	Ultrasonic Whistle	Ultrasound	Ultrasonic Whistle	Acoustic Profile
adv_1_similarity *	Wideband Pulses	Ultrasonic Pulses	Wideband Pulses	Ultrasonic Pulses	
adv_2_similarity *	Vibration Pulses	Wideband Pulses	Vibration Pulses	Wideband Pulses	Valve states
adv_3_similarity *	High Frequency Vibrations	Vibration Pulses	High Frequency Vibrations	Vibration Pulses	

adv_4_similarity *	Rotation Frequency and Harmonics	High Frequency Vibrations	Low Frequency Vibrations	High Frequency Vibrations	
adv_5_similarity *	Rotation Frequency	Rotation Frequency and Harmonics		Low Frequency Vibrations	
adv_5_similarity *	Low Frequency Vibrations	Rotation Frequency			
adv_5_similarity *		Low Frequency Vibrations			

Grey boxes indicate fields that shall be discarded as not applicable for this strategy

Notes	* adv_n_similarity are percentages (in [0;1])
-------	---

Processing very long jobs

When the server returns HTTP Status 202 Accepted, it means that the request has been accepted for processing, but the amount of resulting data is too large to be computed and returned immediately. Instead, for tasks that are likely to take a significant amount of time to be completed, a temporary job identifier is returned. The API user can check periodically whether the resource is ready or not. The returned information is in the following form:

202 Accepted

```
{
  "location": "<job identifier>",
  "datatype": "JobQueue"
}
```

The client can then periodically send GET request to check the availability of the resource using the following keys in the request.

Keys	Values
datatype	JobQueue
location	<job identifier>

Once the request has completed, the HTTP status code 200 OK is returned and the returned payload holds the requested data.

Request example

```
curl https://<domain>/dev/api?datatype=JobQueue&location=f483f9cfaff84f83a
--header "Authorization: Basic <valid authentication string>"
--request GET
```

If long jobs have been started, the server will then return one of two following responses.

Response when the long job is still pending

202 Accepted

```
{
  "job": "f483f9cfaff84f83a ",
  "state": "Pending"
}
```

Response when the long job has completed

200 OK

```
[
  {
    "time": "2025-04-28T13:52:43.354000Z",
```

```

        "device": "00-04-a3-0b-01-07-fb-49",
        "value": 3.376821545738918
    },
    {
        "time": "2025-04-28T13:57:43.250000Z",
        "device": "00-04-a3-0b-01-07-fb-49",
        "value": 3.3752956435492485
    },
    ...
]

```

If no long job has been started, an error will be returned :

410 Gone

```

{
    "error": "This job has either completed or does no longer exist"
}

```

Note 1: The final resource will be purged one hour after job completion.

Note 2: If two same requests (same measurement, same starting date and time) are made within that one hour timeout, the second request will be immediately served with the existing data if available. If the requested data are not available, a 202 Accepted with the same job identifier is returned.

List all devices per organization

This service allows to list all devices part of a given organization. This organization is specified by the Grafana token to be passed as argument.

Endpoint : /devices

Method : GET

Authentication type : HTTP Basic Authentication

Headers

Header name	Value
Authorization	"Basic: " + Base64-encoded credentials string

Query parameters

- *Grafana token*
 - name: grafana_token
 - description: A valid Grafana service account bearer token; refer to Grafana administration manual for more information on how to get such a token
 - type: string
 - required: true

Responses

HTTP status code	Description
200	The list of devices has been returned.
401	Invalid credentials or access token.
400	User input error, the request could not be processed. Check error message for more details.
500	Internal server error, the request could not be processed. Check error message for more details.

Request example

```
curl https://<domain>/dev/devices?grafana_token=xxxxxxxxx
--header "Authorization: Basic xxxxxxxxxxxx"
--request GET
```

Response example

```
{
  "devices": [
    {
      "mac_address": "00-04-a3-0b-01-02-03-04",
      "machine_name": "Internal pump #2"
    },
    {
      "mac_address": "00-04-a3-0b-00-01-02-03",
      "machine_name": "Internal pump #1"
    },
    {
      "mac_address": "00-04-a3-0b-09-08-07-06",
      "machine_name": "External pump 4"
    },
    {
      "mac_address": "02-02-03-04-05-06-07-08",
      "machine_name": "External pump 3"
    }
  ]
}
```

```

    },
    {
      "mac_address": "00-04-a3-0b-00-00-00-01",
      "machine_name": "Reverse motor"
    },
    {
      "mac_address": "00-04-a3-0b-01-01-01-01",
      "machine_name": "Straight motor"
    },
    {
      "mac_address": "00-04-a3-0b-11-22-33-44",
      "machine_name": "Training beacon"
    }
  ],
  "org_id": 1,
  "org_name": "ORG_1"
}

```

Get the identifier of a device attached to a dashboard

This service allows to determine the identifier (DevEUI) of the device associated to a dashboard. The dashboard is specified by its plain name.

Endpoint : /dashboards/<dashboard name>

Method : GET

Authentication type : HTTP Basic Authentication

Headers

Header name	Value
Authorization	"Basic: " + Base64-encoded credentials string

Query parameters

- None

Responses

HTTP status code	Description
200	The device identifier has been returned.
401	Invalid credentials.
400	User input error, the request could not be processed. Check error message for more details.

HTTP status code	Description
500	Internal server error, the request could not be processed. Check error message for more details.

Request example

```
curl https://<domain>/dev/dashboards/my_dashboard
--header "Authorization: Basic xxxxxxxxxxxx"
--request GET
```

Response example

```
[
  "my_dashboard",
  "00-04-a3-0b-01-05-e4-03"
]
```

Get device information by its identifier

This service allows to fetch some information on a device identified by its DevEUI.

The information returned is :

- the number of data points sent over the last 24h
- the date/time of the first data point ever sent by this device
- the date/time of the last data point sent by this device.

Endpoint : /devices/<DevEUI>

Method : GET

Authentication type : HTTP Basic Authentication

Headers

Header name	Value
Authorization	"Basic: " + Base64-encoded credentials string

Query parameters

- None

Responses

HTTP status code	Description
200	The device information has been returned.
401	Invalid credentials.

HTTP status
code

Description

400	User input error, the request could not be processed. Check error message for more details.
500	Internal server error, the request could not be processed. Check error message for more details.

Request example

```
curl https://<domain>/dev/devices/ 00-04-a3-0b-00-fa-73-df
  --header "Authorization: Basic xxxxxxxxxx"
  --request GET
```

Response example

```
{
  "nb_data_points": 284,
  "first_frame_date": "2024-12-11T12:40:11.999Z",
  "last_frame_date": "2026-01-30T07:32:26.481Z"
}
```

UPDATE API

General information

This API provides services to create, update and delete dashboards, and to manage settings and various other information. When creating or updating a dashboard, some textual information can be passed in english or in french, depending on the language of the Grafana organization containing the dashboard.

URL Hook : `https://<server_url>/<endpoint>/...`

Services

Dashboard creation

This service allows to create a new Grafana dashboard. Unless otherly specified, the dashboard is stored in the "General" Grafana folder.

Endpoint : `/dashboard`

Method : `POST`

Authentication type : HTTP Basic Authentication

Headers

Header name	Value
Authorization	"Basic: " + Base64-encoded credentials string
Content-Type	"application/json"

Payload

A stringified JSON object containing the following fields :

- *Machine name*
 - name: machine_name
 - description: Name of the machine on which the beacon is placed
 - type: string (max: 255 characters)
 - required: true
- *MAC address*
 - name: mac_address
 - description: MAC address of the beacon
 - type: valid MAC address, i.e. compliant with the regular expression `/^([0-9a-zA-Z]{2}-){7}[0-9a-zA-Z]{2}$/`
 - required: true

- *Process function*
 - name: process_function
 - description: Function of the machine on which the beacon is placed
 - type: string; one of the following values : “Compressor”, “Conveyor”, “Crusher”, “Elevator”, “Fan”, “Pump”: “Pompe”, “Valve”, “Other” (for english Grafana organizations) or “Compresseur”, “Convoyeur”, “Broyeur”, “Elévateur”, “Ventilateur”, “Pompe”, “Vanne”, “Autre” (for french Grafana organizations)
 - required: true
- *Machine part*
 - name: type
 - description: Part of the machine on which the beacon is placed
 - type: string; one of the following values: “Bearing”, “Gearbox”, “Load”, “Motor”, “Pulley”, “Valve body”, “Pipe” (for english Grafana organizations) or “Palier”, “Réducteur”, “Charge”, “Moteur”, “Poulie”, “Corps de vanne”, “Tuyau” (for french Grafana organizations)
 - required: true
- *Analysis profile*
 - name: analysis_profile
 - description: General type of the machine the beacon is placed on
 - type: string; one of the following values: “Rotating”, “Non-rotating”, “Valve” (for english Grafana organizations) or “Machine tournante”, “Machine non-tournante”, “Vanne” (for french Grafana organizations)
 - required: true
- *Machine power ISO classification*
 - name: machine_class
 - description: ISO classification of the power of the machine on which the beacon is placed
 - type: string; one of the following values: “z1”, “z2”, “z3”, “z4”, “a1”, “a2”, “a3”, “a4”, “b1”, “b2”, “b3”, “b4”, “default”; refer to AsystemView for details
 - required: true
- *Sensor orientation*
 - name: orientation
 - description: Orientation of radial axis and vertical axis of the beacon sensor on the machine; refer to AsystemView User Guide for details
 - type: string; one of the following values: “- ↓ - ☉”, “- ↓ X ☉”, “- ↓ Y ☉”, “- ↓ Z ☉”, “X ↓ - ☉”, “X ↓ X ☉”, “X ↓ Y ☉”, “X ↓ Z ☉”, “Y ↓ - ☉”, “Y ↓ X ☉”, “Y ↓ Y ☉”, “Y ↓ Z ☉”, “Z ↓ - ☉”, “Z ↓ X ☉”, “Z ↓ Y ☉”, “Z ↓ Z ☉”
 - required: true

- *Sensor location*
 - name: sensor_location
 - description: Place on the machine where the beacon is located
 - type: string; one of the following values: “Casing”, “Foundations”, “Input Shaft”, “Output Shaft” (for english Grafana organizations) or “Carter”, “Fondations”, “Arbre d’entrée”, “Arbre de sortie” (for french Grafana organizations)
 - required: false
- *Manufacturer*
 - name: manufacturer
 - description: Name of the manufacturer of the machine on which the beacon is placed
 - type: string (max: 255 characters)
 - required: false
- *Model*
 - name: model
 - description: Model name of the machine on which the beacon is placed
 - type: string (max: 255 characters)
 - required: false
- *Tags*
 - name: tags
 - description: Tags allowing to group beacon dashboards in Grafana
 - type: string; tags must be separated by a comma character, for instance “tag1, tag2, tag3, tag4”
 - required: false
- *Corporate organization identifier*
 - name: org_id
 - description: An identifier of the corporate organization the beacon is allocated to; in current version of AsystomAdvisor, this identifier corresponds to a Grafana folder id If this identifier is omitted, the beacon is searched in the “General” Grafana folder
 - type: positive integer
 - required: false
- *Grafana token*
 - name: grafana_token
 - description: A valid Grafana service account bearer token; refer to Grafana administration manual for more information on how to get such a token
 - type: string

- required: true

Payload example

```
{
  "grafana_token": "xxxxxxxxxxxxxxxxxxxxxxxx",
  "mac_address": "01-02-03-04-05-06-07-08",
  "machine_class": "b1",
  "machine_name": "B site / pump #6",
  "process_function": "Pump",
  "sensor_location": "Foundations",
  "orientation": "- ↓ - ⌚",
  "type": "Motor",
  "analysis_profile": "Rotating",
  "tags": "Drilling site",
  "org_id": 15
}
```

Responses

HTTP status code	Description
201	Dashboard has been successfully created.
401	Invalid credentials or access token.
403	User input error, the dashboard could not be created. Check error message for more details.
500	Internal server error, the dashboard could not be created. Check error message for more details.

Request example

```
curl https://<domain>/dev/dashboard
--header "Content-Type: application/json"
--header "Authorization: Basic xxxxxxxxxxxxxxxxxxxxxxxx"
--request POST
--data '{ "grafana_token": "xxxxxxxxxxxxxxxxxxxxxxxx",
  "mac_address": "01-02-03-04-05-06-07-08",
  "machine_class": "b1", "machine_name": "B site / pump #6",
  "process_function": "Pump",
  "sensor_location": "Foundations", "orientation": "- ↓ - ⌚",
  "type": "Motor", "analysis_profile": "Rotating",
  "tags": "Drilling site, Pumps", "org_id": 15 }'
```

Dashboard update

This service allows to update a Grafana dashboard definition. Any element of a dashboard can be updated except for the MAC address.

Endpoint : /dashboard

Method : PUT

Authentication type : HTTP Basic Authentication

Headers

Header name	Value
Authorization	"Basic: " + Base64-encoded credentials string
Content-Type	"application/json"

Payload

A stringified JSON object containing the following fields :

- *MAC address*
 - name: mac_address
 - description: MAC address of the beacon
 - type: valid MAC address, i.e. compliant with the regular expression `/^([0-9a-zA-Z]{2}-){7}[0-9a-zA-Z]{2}$/`
 - required: true
- *Machine name*
 - name: machine_name
 - description: Name of the machine on which the beacon is placed
 - type: string (max: 255 characters)
 - required: false
- *Process function*
 - name: process_function
 - description: Function of the machine on which the beacon is placed
 - type: string; one of the following values : "Compressor", "Conveyor", "Crusher", "Elevator", "Fan", "Pump": "Pompe", "Valve", "Other" (for english Grafana organizations) or "Compresseur", "Convoyeur", "Broyeur", "Elévateur", "Ventilateur", "Pompe", "Vanne", "Autre" (for french Grafana organizations)
 - required: false
- *Machine part*
 - name: type
 - description: Part of the machine on which the beacon is placed
 - type: string; one of the following values: "Bearing", "Gearbox", "Load", "Motor", "Pulley", "Valve body", "Pipe" (for english Grafana organizations) or "Palier", "Réducteur", "Charge", "Moteur", "Poulie", "Corps de vanne", "Tuyau" (for french Grafana organizations)
 - required: false
- *Analysis profile*

- name: analysis_profile
 - description: General type of the machine the beacon is placed on
 - type: string; one of the following values: "Rotating", "Non-rotating", "Valve" (for english Grafana organizations) or "Machine tournante", "Machine non-tournante", "Vanne" (for french Grafana organizations)
 - required: false
- *Machine power ISO classification*
 - name: machine_class
 - description: ISO classification of the power of the machine on which the beacon is placed
 - type: string; one of the following values: "z1", "z2", "z3", "z4", "a1", "a2", "a3", "a4", "b1", "b2", "b3", "b4", "default"; refer to AsystomView for details
 - required: false
- *Sensor orientation*
 - name: orientation
 - description: Orientation of radial axis and vertical axis of the beacon sensor on the machine; refer to AsystomView User Guide for details
 - type: string; one of the following values: "- ↓ - ☉", "- ↓ X ☉", "- ↓ Y ☉", "- ↓ Z ☉", "X ↓ - ☉", "X ↓ X ☉", "X ↓ Y ☉", "X ↓ Z ☉", "Y ↓ - ☉", "Y ↓ X ☉", "Y ↓ Y ☉", "Y ↓ Z ☉", "Z ↓ - ☉", "Z ↓ X ☉", "Z ↓ Y ☉", "Z ↓ Z ☉"
 - required: false
- *Sensor location*
 - name: sensor_location
 - description: Place on the machine where the beacon is located
 - type: string; one of the following values: "Casing", "Foundations", "Input Shaft", "Output Shaft" (for english Grafana organizations) or "Carter", "Fondations", "Arbre d'entrée", "Arbre de sortie" (for french Grafana organizations)
 - required: false
- *Manufacturer*
 - name: manufacturer
 - description: Name of the manufacturer of the machine on which the beacon is placed
 - type: string (max: 255 characters)
 - required: false
- *Model*
 - name: model
 - description: Model name of the machine on which the beacon is placed
 - type: string (max: 255 characters)

- required: false
- *Tags*
 - name: tags
 - description: Tags allowing to group beacon dashboards in Grafana
 - type: string; tags must be separated by a comma character, for instance “tag1, tag2, tag3, tag4”
 - required: false
- *Corporate organization identifier*
 - name: org_id
 - description: An identifier of the corporate organization the beacon is allocated to; in current version of AsystemAdvisor, this identifier corresponds to a Grafana folder id If this identifier is omitted, the beacon is searched in the “General” Grafana folder
 - type: positive integer
 - required: false
- *Grafana token*
 - name: grafana_token
 - description: A valid Grafana service account bearer token; refer to Grafana administration manual for more information on how to get such a token
 - type: string
 - required: true

Payload example

```
{
  "grafana_token": "xxxxxxxxxxxxxxxxxxxxxxxx",
  "mac_address": "01-02-03-04-05-06-07-08",
  "sensor_location": "Foundations",
  "orientation": "- ↓ - ↻"
}
```

Responses

HTTP status code	Description
200	Dashboard has been successfully updated.
401	Invalid credentials or access token.
403	User input error, the dashboard could not be updated. Check error message for more details.
404	User input error, the dashboard could not be found. Check error message for more details.

HTTP status code	Description
500	Internal server error, the dashboard could not be updated. Check error message for more details.

Request example

```
curl https://<domain>/dev/dashboard
--header "Content-Type: application/json"
--header "Authorization: Basic xxxxxxxxxxxxxxxxxxxxxxxx"
--request PUT
--data '{ "grafana_token": "xxxxxxxxxxxxxxxxxxxxxx",
  "mac_address": "01-02-03-04-05-06-07-01",
  "sensor_location": "Foundations", "orientation": "- ↓ - ☺",
  "tags": "Drilling site, Pumps", "org_id": 15 }'
```

Dashboard deletion

This service allows to delete a Grafana dashboard.

Endpoint : /dashboard

Method : DELETE

Authentication type : HTTP Basic Authentication

Headers

Header name	Value
Authorization	"Basic: " + Base64-encoded credentials string

Query parameters

- *Grafana token*
 - name: grafana_token
 - description: A valid Grafana service account bearer token; refer to Grafana administration manual for more information on how to get such a token
 - type: string
 - required: true
- *MAC address*
 - name: mac_address
 - description: MAC address of the beacon (aka. DevEUI)
 - type: valid MAC address, i.e. compliant with the regular expression `/^([0-9a-zA-Z]{2}-){7}[0-9a-zA-Z]{2}$/`
 - required: true

Request example

```
curl
https://<domain>/dev/dashboard?grafana_token=xxxxxxxxx&mac_address=yyyy
YYYYYYYYYY
--header "Authorization: Basic xxxxxxxxxxxx"
--request DELETE
```

Responses

HTTP status code	Description
204	Dashboard has been successfully deleted.
401	Invalid credentials or access token.
404	User input error, the dashboard could not be found. Check error message for more details.
500	Internal server error, the dashboard could not be updated. Check error message for more details.

Dashboard settings change

This service allows to register the new settings to be applied on a beacon. Once registered, the settings will be applied on the selected beacon the next time the beacon will transmit data to a gateway. Shortly after having sent its data for uplink, the beacon will listen for a few seconds to potential orders from the gateway; the gateway will use this time slice to transmit the new settings. Upon reception of the settings, the beacon take them into account and reboot to acknowledge their reception.

Beacon settings are split in three parts :

- scheduling and activation settings; these allow to activate/deactivate some equipment such as temperature and humidity sensors, and to specify the scheduling of data uplink
- signature settings; the synchronization mode (wake-on-contact, make-on-scheduling, wake-on-motion, etc.) can be chosen this way, as well as RPM- and LoRaWAN-specific settings
- FFT zoom settings; the periodicity of acquisition, spectrum for each sensor (accelerometer and microphone), etc., can be selected this way.

This service allows to specify a value for one of these parts. The selection of which settings part is to be changed is made based on the 'Settings part' element of the request body. See AsystomAdvisor User Guide for more details on the numerous available settings.

The settings are specified as a hex string. The hex string structure is described in document Payload_Decompression.pdf. A hex string can be generated using a dedicated tool on Asystom GitHub at <https://asystom-iot.github.io/beacon-settings-generator/>.

In addition to beacon settings change, this service allows to restart a beacon. In that case, the hex string representing the new settings is not taken into account.

Asystom supports various LoRaWAN network configurations and LoRaWAN network server (LNS) configurations :

- beacons managed by gateways connected to an LNS that can receive settings change requests from the AsystomAdvisor back-end (such as Kerlink WMC and Loriot LNSs for instance)
- beacons managed by gateways connected to an LNS that fetches settings change data from the AsystomAdvisor back-end (some Tektelic or Actility LNSs seem to work this way)
- beacons managed by a single gateway embedding an LNS such as Chirpstack, or embedding their own LNS (Kerlink SPN gateways and some Multitech gateway models work like this).

To handle settings change in these various use cases, the AsystomAdvisor back-end needs some additional information : the LNS identifier and the LoRaWAN gateway identifier. This information allows to select the right network server and possibly gateway to which the configuration settings should be attached. The gateway identifier is used only for POC contexts where a gateway embeds the LNS; in this situation, the gateway identifier is required. Otherwise, the “gateway” argument should hold the same value as the “client” argument.

The LNS identifier is a string that is attached to uplink frames. Depending on the LNS and customer context, it may be placed in the body of the HTTP POST request used to transmit the uplink frames or in a header of this request.

Endpoint : /beacon_settings

Method : POST

Authentication type : HTTP Basic Authentication

Headers

Header name	Value
Authorization	“Basic: ” + Base64-encoded credentials string
Content-Type	“application/json”

Payload

A stringified JSON object containing the following fields :

- *MAC address*
 - name: mac_address
 - description: MAC address of the beacon

- type: valid MAC address, i.e. compliant with the regular expression `/^([0-9a-zA-Z]{2}-){7}[0-9a-zA-Z]{2}$/`
 - required: true
- *LoRaWAN Client identifier*
 - name: client
 - description: Identifier of the LoRaWAN network server
 - type: a string representing the LoRaWAN network server identifier
 - required: true
- *LoRaWAN gateway identifier*
 - name: gateway
 - description: Identifier of a LoRaWAN gateway
 - type: a string representing the LoRaWAN gateway identifier
 - required: true
- *Settings part selection*
 - name: command
 - description: Part of the settings to be modified
 - type: integer holding one of the following values: 64 (simple beacon reboot), 70 (scheduling change), 71 (signature change), 74 (FFT zoom settings change)
 - required: true
- *Settings value*
 - name: settings
 - description: New settings to be applied on the beacon
 - type: a hex string representing the settings to be applied
 - required: true

Payload example

```
{
  "mac_address": "01-02-03-04-05-06-07-08",
  "client": "ACME network",
  "gateway": "ACME network",
  "command": 74,
  "settings": "3010A000300D00700000000000000400000001000000C8000000"
}
```

Responses

HTTP

status code	Description
-------------	-------------

200	Settings have been successfully registered.
401	Invalid credentials.
403	User input error, the settings could not be registered. Check error message for more details.
404	User input error, the dashboard could not be found. Check error message for more details.
500	Internal server error, the settings could not be registered. Check error message for more details.

Request example

```
curl https://<domain>/dev/beacon_settings
--header "Content-Type: application/json"
--header "Authorization: Basic xxxxxxxxxxxxxxxxxxxxxxxx"
--request POST
--data '{ "mac_address": "01-02-03-04-05-06-07-01",
          "client": "ACME network",
          "gateway": "ACME network",
          "command": 71,
          "settings": "0C00000094110000D00700001C001600050032000100C8
00110090010000000049000000100E" }'
```